

Kubernetes v1.36: Haru



kubernetes



Ryota Sawada

v1.36 Release Lead
Software Engineer, Numtide



Josh Michielsen

v1.36 Enhancements Lead
Sr. Principal Engineer, Dataminr



Chad M. Crowell

v1.36 Comms Lead
Principal SRE, Akamai



Kubernetes v1.36 Release Team

Release Lead & Shadows

Ryota Sawada
Agustina Barbetta
Dipesh Rawat
Jenny Shu
Rayan Das

Branch Management

Sreeram Venkitesh
Drew Hagen
Agustina Barbetta
Michelle Nguyen

Release Team Subproject Lead

Kat Cosgrove
Frederico Munoz

Enhancements

Josh Michielsen
ChengHao Yang
Lauri Apple
Peter Engelbert
Rajalakshmi Girish
Subhasmita Swain

Release Signal

Prajyot Parab
Aman Shrivastava
Dhanisha Phadate
Graziano Casto
Keisuke Ishigami
Muhammad Adil Ghaffar
Tatiana Selezneva

Docs

Sayan Chowdhury
Anshuman Tripathi
Emile Savard
Saurabh Kumar Singh
Tushar Mittal

Comms

Chad M. Crowell
Swathi Rao
Kirti Goyal
Sophia Ugochukwu
Swathi Rao
Utkarsh Umre



Kubernetes v1.36: ハル (Haru)

ハル (Haru) is a sound in Japanese that carries many meanings: 春 (*haru*, spring), 晴れ (*hare*, clear skies), and 遥か (*haruka*, far-off, distant). A season, a sky, and a horizon.

Logo, created by [avocadoneko / Natsuho Ide](#), draws inspiration from Katsushika Hokusai's [Fine Wind, Clear Morning](#) (凱風快晴, *Gaifū Kaisei*), also known as Red Fuji (赤富士, *Aka Fuji*).

晴れに翔け、未来よ明け

hare ni kake, asu yo ake

"Soar into clear skies; toward tomorrow's sunrise."



Summary of Enhancements

70 Enhancements in v1.36

- 25 enhancements to **Alpha**
- 25 enhancements to **Beta**
- 18 enhancements to **Stable**

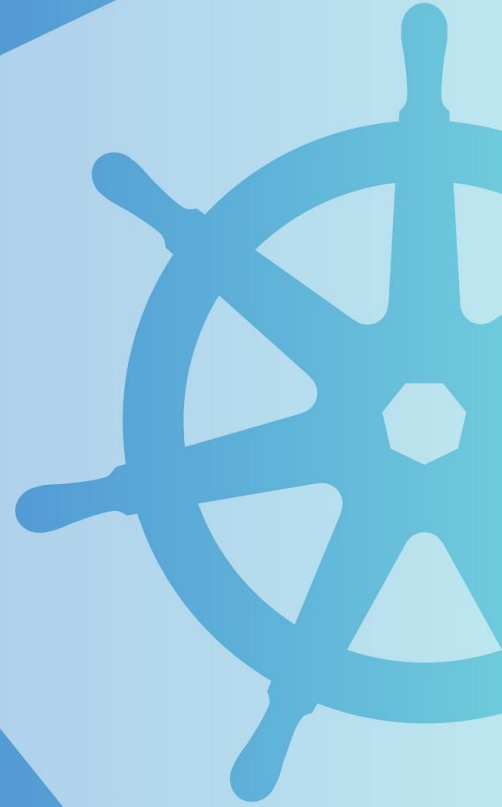
- 2 features **Deprecated**

For the complete list of enhancements:

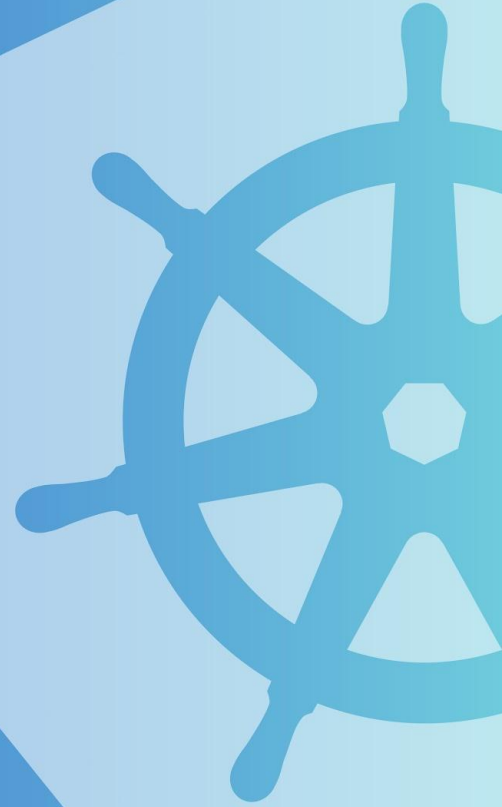
<https://github.com/orgs/kubernetes/projects/241/views/1>



Release Highlights



SIG Scheduling



Decoupled PodGroup API

The Workload API now serves as a static template, while the new PodGroup API describes the runtime object. Introduced as part of the scheduling.k8s.io/v1alpha2 API group, completely replacing the previous v1alpha1 API version.

In v1.35, Pod groups and their runtime states were embedded within the Workload resource. The new model decouples these concepts: the Workload now serves as a static template object, while the PodGroup manages the runtime state. This separation also improves performance and scalability as the PodGroup API allows per-replica sharding of status updates.



Status: Alpha

- Alpha: v1.36

KEP

[KEP-5832: WAS: Decouple PodGroup API](https://kep-5832.was.dev/decouple-podgroup-api)
(features.k8s.io/5832)



Topology-aware Workload Scheduling

For complex distributed workloads like AI/ML training or batch processing, placing Pods randomly across a cluster can introduce significant network latency and bottleneck overall performance.

Topology-aware scheduling addresses this problem by allowing you to define topology constraints directly on a PodGroup, ensuring its Pods are co-located within specific physical or logical domains

Status: Alpha

○ Alpha: v1.36



KEP

[KEP-5732: Topology-aware Workload Scheduling](https://kubernetes.io/features/5732/)
(features.k8s.io/5732)



Workload-aware Preemption

Kubernetes v1.36 introduces a new type of preemption mechanism called workload-aware preemption. When a PodGroup cannot be scheduled, the scheduler utilizes this mechanism to try making a scheduling of this PodGroup possible.

This new mechanism treats the entire PodGroup as a single preemptor unit. Instead of evaluating preemption victims on each Node separately, it searches across the entire cluster. This allows the scheduler to preempt Pods from multiple Nodes simultaneously, making enough space to schedule the whole PodGroup afterwards.

Status: Alpha

- Alpha: v1.36

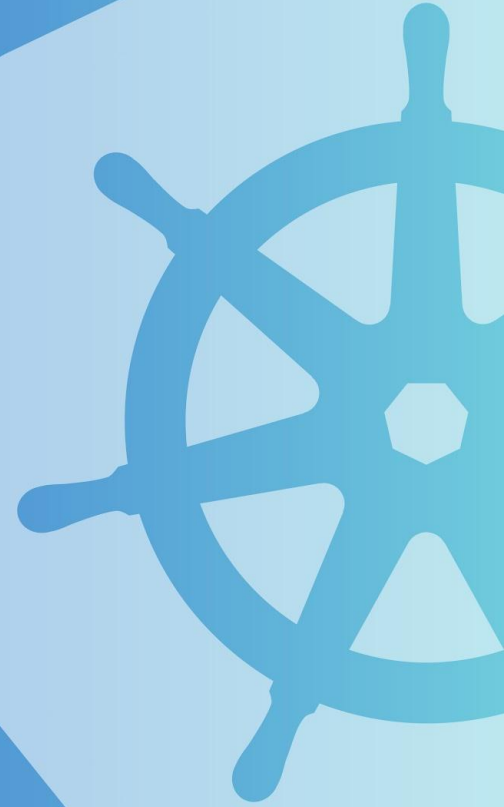


KEP

[KEP-5710: Workload-aware Preemption](#)
(features.k8s.io/5710)



WG Device Management



DRA: AdminAccess for ResourceClaims and ResourceClaimTemplates

Graduating to Stable in Kubernetes v1.36, DRA AdminAccess allows administrators to securely access devices already in use by other users.

This enables administrators to perform tasks such as monitoring the health and status of all devices while ensuring that regular users can only run containers on devices explicitly claimed by their workloads.

Status: **Stable**

- Alpha: v1.33
- Beta: v1.34
- Stable: v1.36

KEP

[KEP-5018: DRA: AdminAccess for ResourceClaims and ResourceClaimTemplates](#)
(features.k8s.io/5018)



DRA: Prioritized Alternatives in Device Requests

The Prioritized List DRA feature graduates to Stable, allowing users to define fallback preferences when requesting devices.

Rather than hardcoding a request for a specific device model, users can instead specify an ordered list of preferences (e.g., "Give me an H100, but if none are available, fall back to an A100"). Preferences will be evaluated in order, drastically improving scheduling flexibility and cluster utilization.

Status: **Stable**

- Alpha: v1.33
- Beta: v1.34
- Stable: v1.36



KEP

[KEP-4816: DRA: Prioritized Alternatives in Device Requests](#)
(features.k8s.io/4816)



DRA: Partitionable Devices

The Partitionable Devices feature provides native support for dynamically carving physical hardware into small, logical instances.

This enables users to more efficiently share expensive hardware resources across multiple workloads in cases where a single workload doesn't need a full device to operate.



Status: Beta

- Alpha: v1.35
- Beta: v1.36

KEP

[KEP-4815: DRA: Partitionable Devices](#)

(features.k8s.io/4815)





DRA: Device Taints and Tolerations

Device Taints and Tolerations graduates to Beta this release. Just as you can taint a Kubernetes Node, you can also apply taints directly to specific DRA devices.

This empowers administrators to manage hardware more effectively, allowing you to taint faulty devices, or reserve specific hardware for dedicated teams, specialized workloads, and experiments. Ultimately, only Pods with matching tolerations are permitted to claim these tainted devices.

Status: Beta

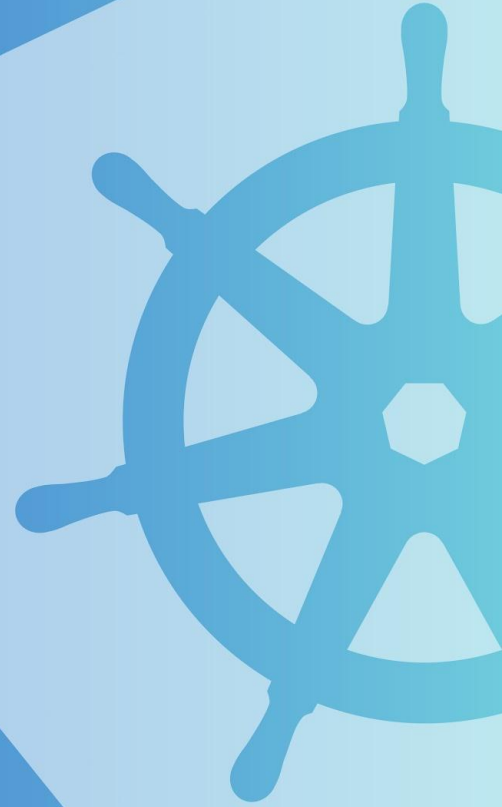
- Alpha: v1.33
- Beta: v1.36

KEP

[KEP-5055: DRA: Device Taints and Tolerations](#)
(features.k8s.io/5055)



SIG Apps



Mutable container resources when Job is suspended

`MutablePodResourcesForSuspendedJobs` feature graduates to beta and is enabled by default. This update relaxes Job validation to allow updates to container CPU, memory, GPU, and extended resource requests and limits while a Job is suspended.

For example, a queueing system can suspend incoming Jobs, adjust their resource requirements to match available capacity or quota, and then unsuspend them.



Status: Beta

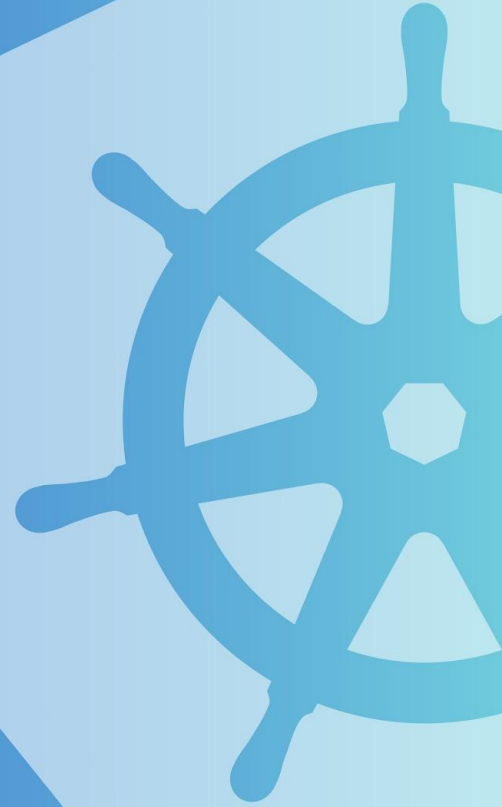
- Alpha: v1.35
- Beta: v1.36

KEP

[KEP-5440: Mutable Container Resources when Job is suspended](#)
(features.k8s.io/5440)



SIG Windows



Node Log Query

Previously, Kubernetes required cluster administrators to log into nodes via SSH or implement a client-side reader for debugging issues pertaining to control-plane or worker nodes.

Node logs offer a method to view these logs using the kubelet API and kubectl plugin to simplify troubleshooting without logging into nodes, similar to debugging issues related to a pod or container. This method is operating system agnostic and requires the services or nodes to log to `/var/log`.



Status: **Stable**

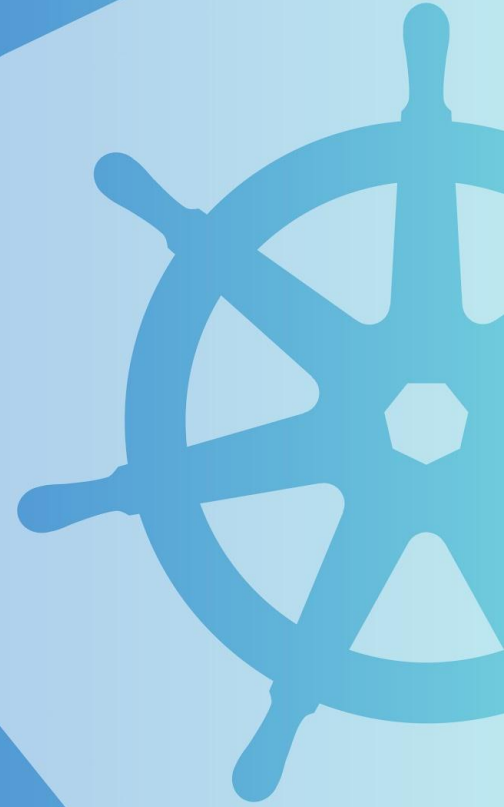
- Alpha: v1.27
- Beta: v1.30
- Stable: v1.36

KEP

[KEP-2258: Node log query](https://features.k8s.io/2258)
(features.k8s.io/2258)



SIG Instrumentation



Statusz and Flagz for components

`ComponentStatusz` and `ComponentFlagz` feature gates for core Kubernetes components graduate to beta, providing `/statusz` and `/flagz` endpoints respectively. They offer a human-readable text view by default, plus a versioned structured API for programmatic access, with access granted to `system:monitoring` group.

With beta, `ComponentStatusz` and `ComponentFlagz` are both enabled by default and implemented across all core control-plane components and node agents.

Status: Beta

- Alpha: v1.32
- Beta: v1.36

KEP

- [KEP-4827: Statusz for Kubernetes Components](https://features.k8s.io/4827) (features.k8s.io/4827)
- [KEP-4828: Flagz for Kubernetes Components](https://features.k8s.io/4828) (features.k8s.io/4828)



Native histogram support for Kubernetes metrics

High-resolution monitoring reaches a new milestone in Kubernetes v1.36 with the introduction of native histogram support in Alpha.

Now, cluster operators can capture precise latency distributions for the kube-apiserver and other core components without the overhead of manual bucket management. This architectural shift ensures more reliable SLIs and SLOs, providing high-fidelity heatmaps that remain accurate even during the most unpredictable workload spikes.



Status: Alpha

- Alpha: v1.36

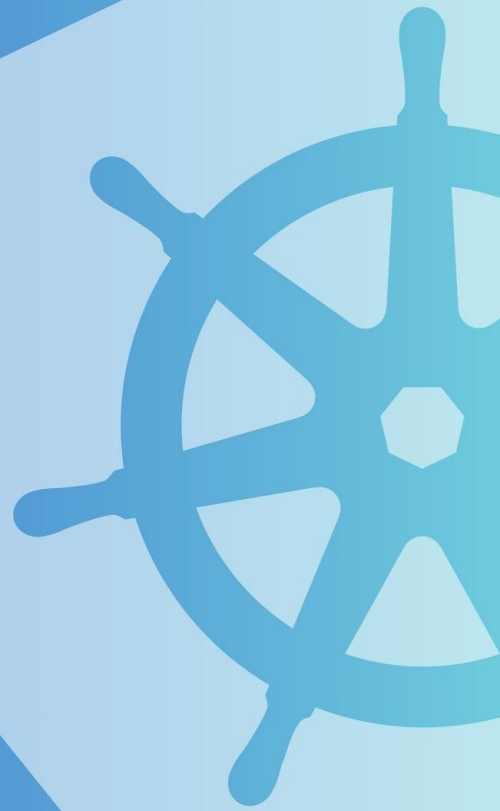
KEP

[KEP-5808: Native Histogram Support for Kubernetes Metrics](#)

[\(features.k8s.io/5808\)](https://features.k8s.io/5808)



SIG Storage



Volume Group Snapshots

After several cycles in beta, `VolumeGroupSnapshot` support reaches General Availability (GA) in Kubernetes v1.36.

This feature allows you to take crash-consistent snapshots across multiple `PersistentVolumeClaims` simultaneously. The support for volume group snapshots relies on a set of extension APIs for group snapshots. These APIs allow users to take crash consistent snapshots for a set of volumes. A key aim is to allow you to restore that set of snapshots to new volumes and recover your workload based on a crash consistent recovery point.



Status: **Stable**

- Alpha: v1.27
- Beta: v1.32
- Beta2: v1.34
- Stable: v1.36

KEP

[KEP-3476:](#)

[VolumeGroupSnapshot](#)
(features.k8s.io/3476)



Mutable Volume Attach Limits

The mutable `CSINode` allocatable feature graduates to stable. This enhancement allows Container Storage Interface (CSI) drivers to dynamically update the reported maximum number of volumes that a node can handle.

kubelet can now dynamically update a node's volume limits and capacity information, adjust these limits based on periodic checks or in response to resource exhaustion errors from the CSI driver, without requiring a component restart. The scheduler can maintain an accurate view of storage availability, preventing pod scheduling failures caused by outdated volume limits.



Status: **Stable**

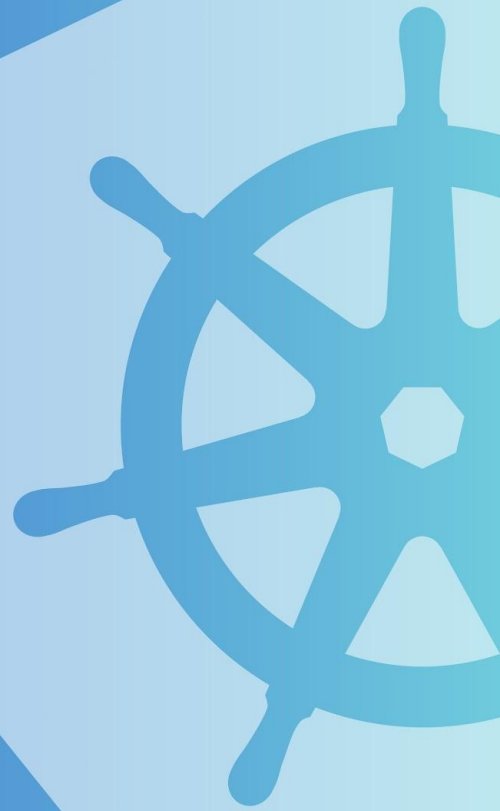
- Alpha: v1.33
- Beta: v1.34
- Stable: v1.36

KEP

[KEP-4876: Mutable CSINode Allocatable Property](#)
(features.k8s.io/4876)



SIG CLI



Separate kubectl user preferences from cluster configs

The `.kuberc` feature for customizing `kubectl` user preferences continues to be beta and is enabled by default.

In Kubernetes v1.36, `.kuberc` was expanded with the ability to define policies for credential plugins (allowlists or denylists) to enforce safer authentication practices.

Status: Beta

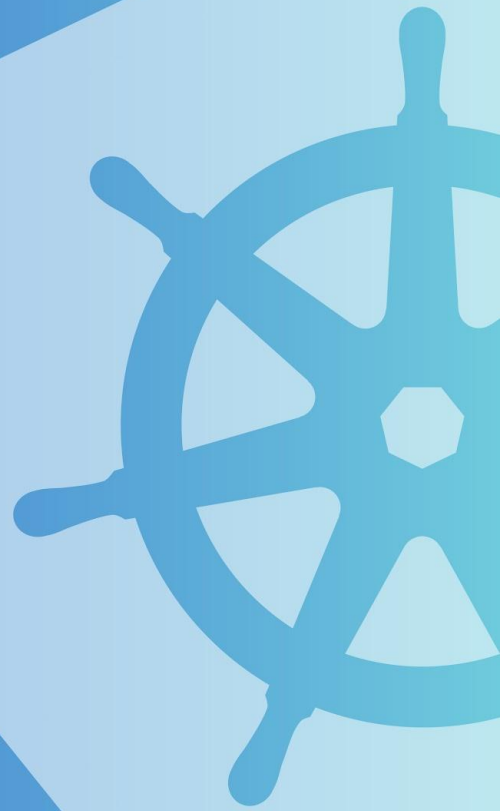
- Alpha: v1.33
- Beta: v1.34, v1.35, v1.36

KEP

[KEP-3104: Separate kubectl user preferences from cluster configs](#)
(features.k8s.io/3104)



SIG Auth



Fine-grained API authorization

Currently, the kubelet API authorization uses a coarse auth scheme, where actions like reading health status and ability to exec into pods require the same RBAC permissions.

This feature enables more precise, least-privilege access control over the kubelet's HTTPS API replacing the need to grant overly broad permissions for common monitoring and logging agents.



Status: **Stable**

- Alpha: v1.32
- Beta: v1.33
- Stable: v1.36

KEP

[KEP-2862: Fine-grained Kubelet API Authorization](#)
(features.k8s.io/2862)



API for external signing of ServiceAccount tokens

Traditionally, `serviceAccount` keys are stored on the disk and loaded by kube-apiserver during process start time. Since keys are entirely within the cluster means its less flexible because the signing keys remain the same for the lifetime of the process.

This KEP allows Kubernetes distributions to integrate with key management solutions of their choice (e.g. HSMs, cloud KMSes)

Status: **Stable**

- Alpha: v1.32
- Beta: v1.34
- Stable: v1.36

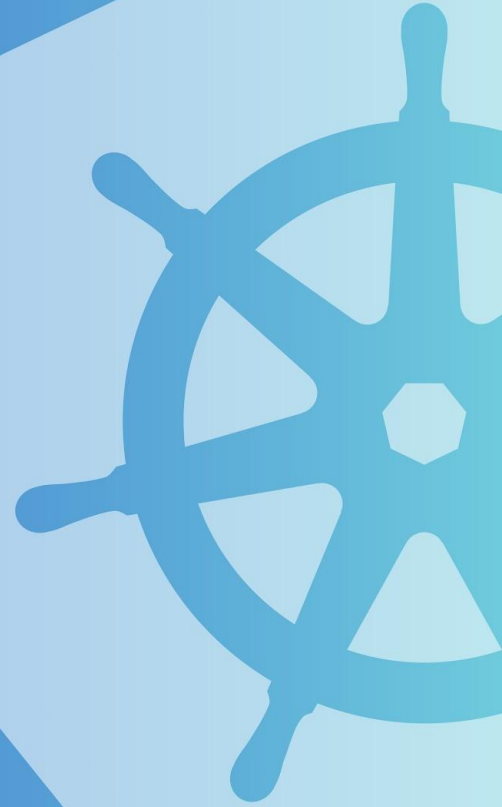
KEP

[KEP-740: API for external signing of Service Account tokens](#)

(<https://kep.k8s.io/740>)



SIG Network



IP/CIDR validation improvements

Traditionally, validation of IP addresses and CIDR strings in API fields has been too lax, accepting any string accepted by the underlying functions, even though in some cases these strings had ambiguous meanings that could lead to security problems.

The `StrictIPCIDRValidation` feature in v1.36 tightens validation to catch malformed addresses and prefixed that previously slipped through.

Status: Beta

- Alpha: v1.33
- Beta: v1.36

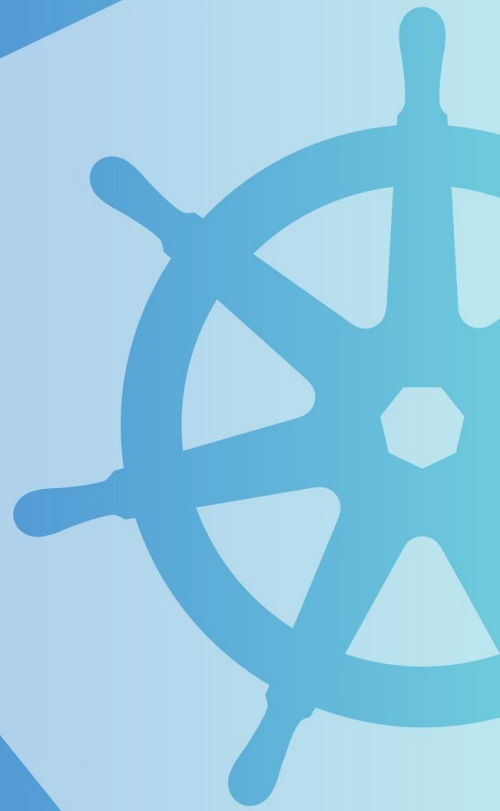
KEP

[KEP-4858: IP/CIDR Validation Improvements](https://kep.k8s.io/4858)

(<https://kep.k8s.io/4858>)



SIG Autoscaling



HPA scale to zero for custom metrics

Horizontal Pod Autoscaler (HPA) automatically scales the number of pods in any resource which supports the `scale` subresource based on observed CPU or memory utilization from one to many replicas.

This proposal adds support for scaling from zero to many replicas and back to zero for object and external metrics.

Status: Alpha

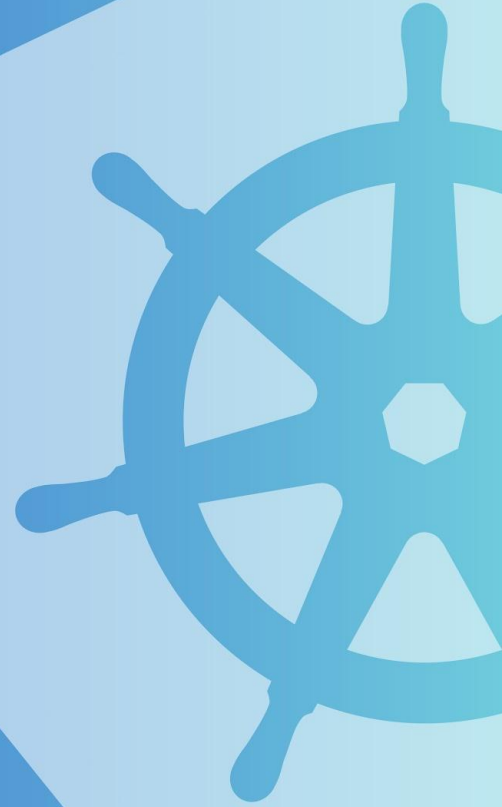
- Alpha: v1.16
- Alpha2: v1.36

KEP

[KEP-2021: Support scaling to/from zero pods for object/external metrics](https://kep.k8s.io/2021)
(<https://kep.k8s.io/2021>)



SIG Cloud Provider



Convert route controllers to watch-based

The route controller in the cloud-controller-manager is currently reconciling in a fixed interval (default: 10s). This leads to unnecessary API requests towards the infrastructure providers.

In v1.36, the reconciler will switch to a watch-based mechanism, so reconciliation would happen whenever there is a node added, removed or certain fields are updated.

Status: Alpha

- Alpha: v1.35
- Alpha2: v1.36

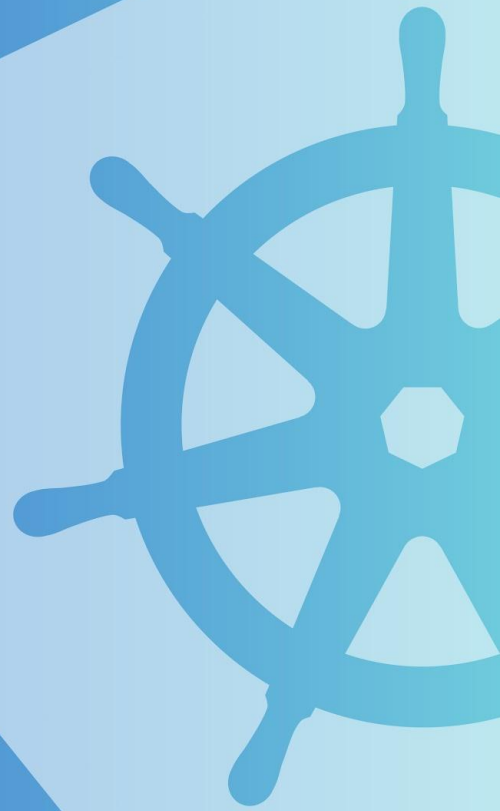
KEP

[KEP-5237: watch-based route controller reconciliation using informers](https://kep.k8s.io/5237)

(<https://kep.k8s.io/5237>)



SIG API Machinery



Mutating Admission Policies

Graduating to GA in Kubernetes v1.36 - Mutating Admission Policies provides a native, high-performance alternative to traditional webhooks by allowing administrators to define resource mutations directly in the API server using the Common Expression Language (CEL), fully replacing the need for external infrastructure for many common use cases.

Now, cluster operators can modify incoming requests without the latency and operational complexity associated with managing custom admission webhooks.



Status: **Stable**

- Alpha: v1.32
- Beta: v1.34
- Stable: v1.36

KEP

[KEP-3962: Mutating Admission Policies](#)
(features.k8s.io/3962)



Stale Controller Mitigation

When operating at a large scale, controller caches can fall behind the state of the apiserver, leading to decisions based on outdated information. This could lead to conflicting updates or data corruption upon controller reconciliation during times of cache staleness.

Kubernetes v1.36 includes new features that help mitigate controller staleness and provide better observability of controller behavior. This prevents reconciliation based on an outdated view of cluster state that can often lead to harmful behavior.



Status: Beta

○ Beta: v1.36

KEP

[KEP-5647: Stale Controller Mitigation](#)

(features.k8s.io/5647)



Manifest Based Admission Control Config

Manifest-based admission control configuration addresses the long-standing challenge of configuring admission plugins through disparate command-line flags or separate, complex config files by allowing administrators to define the desired state of admission control directly through a structured manifest.

Now, cluster operators can manage admission plugin settings with the same versioned, declarative workflows used for other Kubernetes objects, significantly reducing the risk of configuration drift and manual errors during cluster upgrades.

Status: Alpha

- Alpha: v1.36

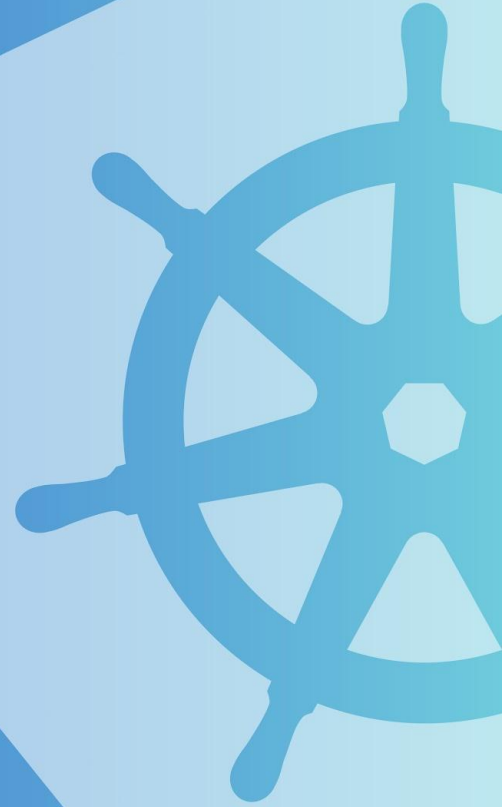


KEP

[KEP-5793: Manifest Based Admission Control Config](#)
(features.k8s.io/5793)



SIG Node



Support User Namespaces in Pods

Container isolation and node security reach a major maturity milestone in Kubernetes v1.36 as support for User Namespaces graduates to Stable.

This provides a critical layer of defense-in-depth by allowing the mapping of a container's root user to a non-privileged user on the host, ensuring that even if a process escapes the container, it possesses no administrative power over the underlying node.

Status: **Stable**

- Alpha: v1.25
- Beta: v1.30 & v1.33
- Stable: v1.36

KEP

[KEP-127: Support User Namespaces in Pods](#)
(features.k8s.io/127)



Volume Source: OCI Artifact and/or Image



The distribution of container data becomes more flexible in Kubernetes v1.36 as OCI volume source support graduates to Stable.

Previously, the distribution of container data was limited to mounting external storage provider volume and ConfigMaps.

Now, developers and platform engineers can package application data, models, or static assets as OCI artifacts and deliver them to pods using the same registries and versioning workflows they already use for container images.

Status: **Stable**

- Alpha: v1.31
- Beta: v1.34
- Stable: v1.36

KEP

[KEP-4639: VolumeSource:
OCI Artifact and/or Image
\(\[features.k8s.io/4639\]\(https://features.k8s.io/4639\)\)](#)



Support PSI based on cgroupv2

Pressure Stall Information (PSI) support provides the kubelet with the ability to report "pressure" metrics for CPU, memory, and I/O, offering a more granular view of resource contention than traditional utilization metrics.

By leveraging these signals, users running cgroup v2 on Linux kernel 4.20 or later with PSI enabled have a stable, reliable interface to observe resource contention at the node, pod, and container levels.



Status: **Stable**

- Alpha: v1.33
- Beta: v1.34
- Stable: v1.36

KEP

[KEP-4205: Support PSI based on cgroupv2](#)
(features.k8s.io/4205)



Support Memory QoS with cgroups v2

Memory QoS uses the cgroup v2 memory controller to give the kernel better guidance on how to treat container memory. First introduced in v1.22 and updated in v1.27, Kubernetes v1.36 introduces support for opt-in memory reservation, tiered protection by QoS class, observability metrics, and kernel-version warning for `memory.high`.

Status: Alpha

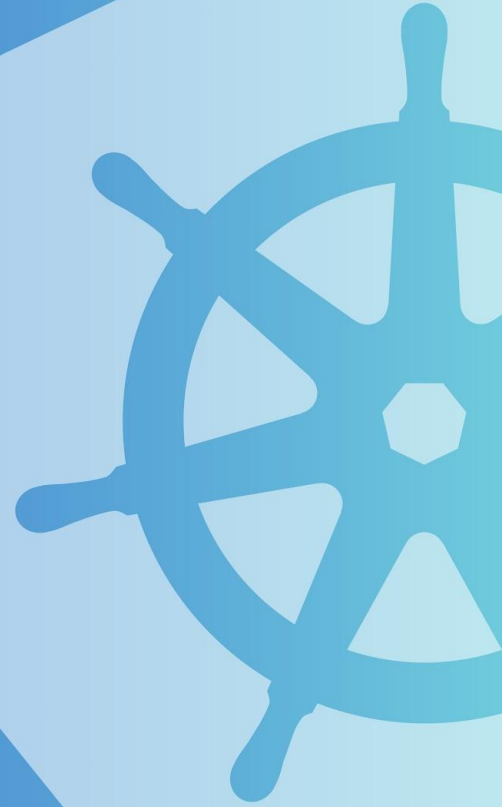
- Alpha 1: v1.22
- Alpha 2: v1.27
- Alpha 3: v1.36

KEP

[KEP-2570: Support Memory QoS with cgroup v2](#)
(features.k8s.io/2570)



Deprecations & Removals



Removal of gitRepo volume driver

The gitRepo volume type has been deprecated since v1.11, but in v1.36 the gitRepo volume plugin is permanently disabled and cannot be turned back on.

This change protects clusters from a critical security issue where using gitRepo could let an attacker run code as root on the node. Any existing workloads depending on gitRepo will need to migrate to supported approaches such as init containers or external git-sync style tools.



KEP

[KEP-5040: Remove gitRepo volume driver](#)

(<https://kep.k8s.io/5040>)



Deprecation of service.spec.externalIPs



The `Service.spec.externalIPs[ ]` field was introduced early in Kubernetes history as a way to expose services on specific external IP addresses.

However, this feature has been considered a security risk and architectural problem, as it allows non-privileged users to claim arbitrary IP addresses without proper authorization or validation, potentially enabling man-in-the-middle attacks and other security exploits.

KEP

[KEP-5707: Deprecate service.spec.externalIPs](https://kep.k8s.io/5707)
(<https://kep.k8s.io/5707>)



Ingress NGINX retirement

To prioritize the safety and security of the ecosystem, Kubernetes SIG Network and the Security Response Committee have retired Ingress NGINX on **March 24, 2026**. Since that date, there have been no further releases, no bugfixes, and no updates to resolve any security vulnerabilities discovered. Existing deployments of Ingress NGINX will continue to function, and installation artifacts like Helm charts and container images will remain available



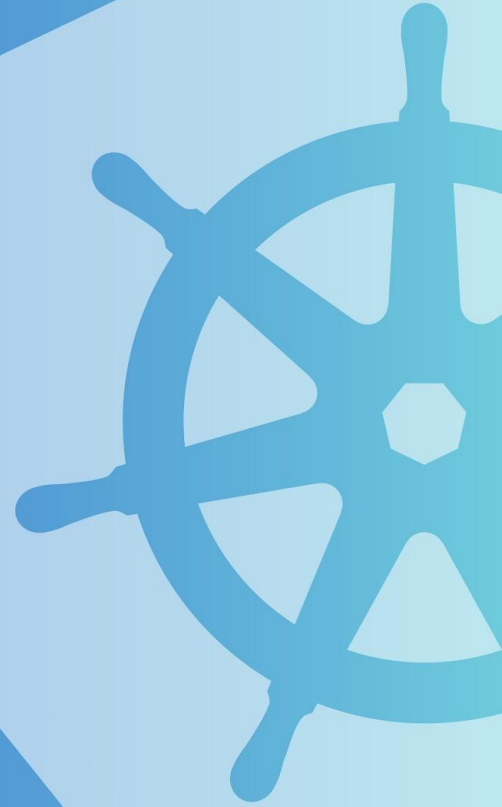
Announcement

[Ingress NGINX Retirement: What You Need to Know](https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/)

(<https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>)



Release Team



Release Team Updates

- v1.36 had over 100+ KEPs originally tracked when the release cycle started. The final KEP count of 70 is highest we've seen, and the release team has successfully conducted all of the actions to have a smooth release out
- There were 17 feature blogs published for v1.36 release cycle. Available at kubernetes.io/blog.
- During the v1.36 release cycle, Kubernetes received contributions from as many as 106 different companies and 491 individuals.





Recap

Release Notes

<https://github.com/kubernetes/kubernetes/blob/master/CHANGELOG/CHANGELOG-1.36.md>

Release Announcement Blog

<https://kubernetes.io/blog/2026/04/22/kubernetes-v1-36-release/>

Slide deck

<https://github.com/kubernetes/sig-release/tree/master/releases/release-1.36>



Get involved

- The simplest way to get involved with Kubernetes is by joining one of the many [Special Interest Groups \(SIGs\)](#) that align with your interests
- Learn more about the [Kubernetes Release Team](#)
- Follow **#sig-release** on [Slack](#), or our [Kubernetes mailing list](#) to get updates on the next Kubernetes Release Team Shadow Application





Thanks!



Q&A

